Partial Classes in C#

1. What is a Partial Class?

A **Partial Class** allows a class definition to be **split across multiple files**.

All the pieces are combined by the compiler into **one single class** during compilation.

Syntax

```
public partial class ClassName
{
   // members
}
```

The keyword **partial** tells the compiler:

"This class definition is incomplete. Other parts of this class exist somewhere else."

---

2. Why Partial Classes Are Used

Partial classes are mainly used to:

1️⃣ **Separate large classes**

If a class becomes very large, it can be split across files.

Example:

Employee_Personal.cs

Employee_Salary.cs

Employee_Operations.cs

This improves **code readability and maintainability**.

2️⃣ **Separate auto-generated code**

Frameworks like:

- **ASP.NET**

- **Entity Framework**
- **Windows Forms**

generate code automatically.

If everything was in one file, developer changes might be **overwritten when code regenerates**.

So Microsoft splits them:

Form1.Designer.cs (auto generated)

Form1.cs (developer code)

Both together form **one class**.

---

3. Example of Partial Class

File 1: Employee_Personal.cs

```
public partial class Employee
{
    public int Id;
    public string Name;

    public void DisplayPersonalInfo()
    {
        Console.WriteLine($"Id: {Id}");
        Console.WriteLine($"Name: {Name}");
    }
}
```

File 2: Employee_Salary.cs

```
public partial class Employee
{
```

```
    public double Salary;

    public void DisplaySalary()
    {
        Console.WriteLine($"Salary: {Salary}");
    }
}
```

Main Program

```
class Program
{
    static void Main()
    {
        Employee emp = new Employee();

        emp.Id = 101;
        emp.Name = "John";
        emp.Salary = 50000;

        emp.DisplayPersonalInfo();
        emp.DisplaySalary();
    }
}
```

Compiler View

Although we wrote two files, the compiler treats it as:

```
public class Employee
```

```
{
    public int Id;
    public string Name;
    public double Salary;

    public void DisplayPersonalInfo() { }

    public void DisplaySalary() { }
}
```

---

4. Rules of Partial Classes

Important rules you should remember:

Rule 1

All parts must use the **partial keyword**.

✓ Correct

```
public partial class Employee
```

❌ Wrong

```
public class Employee
```

Rule 2

All parts must have **same class name**.

✓ Correct

```
partial class Employee
partial class Employee
```

❌ Wrong

```
partial class Employee
```

partial class EmployeeDetails

Rule 3

All parts must be in the **same namespace**.

✓ Correct

```
namespace Company
{
   public partial class Employee { }
}
```

Rule 4

All parts are combined into **one class at compile time**.

---

5. Partial Methods

Partial classes support **Partial Methods**.

These are methods where:

- Declaration in one file
- Implementation in another file

Example

File 1

```
public partial class Employee
{
   partial void DisplayMessage();
}
```

File 2

```
public partial class Employee
```

```
{
    partial void DisplayMessage()
    {
        Console.WriteLine("Hello Employee");
    }
}
```

---

6. Real Example in ASP.NET

If you open an ASP.NET page you will see:

Home.aspx.cs

```
public partial class Home : Page
{
    protected void Page_Load(object sender, EventArgs e)
    {


    }
}
```

Home.aspx.designer.cs

```
public partial class Home
{
    protected global::System.Web.UI.WebControls.Label Label1;
}
```

Both combine to form **one class**.

---

7. Advantages of Partial Classes

✓ Improves **code organization**
✓ Useful for **large applications**
✓ Helps **separate generated code and developer code**
✓ Improves **team development** (multiple developers can work on the same class)

---

8. When Should You Use Partial Classes?

Use when:

- A class is **very large**
- Code is **auto-generated**
- Multiple developers work on the same class
- You want **logical separation of functionality**